

Fuzzing for Web Browser Under-Invalidation Bugs

Anonymous Submission

Abstract—Fuzzers are widely used to find security-related bugs in web browser parsers and media engines through crash testing, and to discover cross-browser and cross-version regressions via differential testing. We introduce a new type of browser correctness bug, under-invalidation, which occurs when a browser incorrectly updates its internal state after a style update. Under-invalidation bugs allow for *self-differential* testing, which tests a browser rendering engine against itself, leading to particularly clean fuzzing results.

To find under-invalidation bugs efficiently, we develop LQC, a fuzzer that rapidly generates candidates web pages, tests them in a real browser via self-differential testing, and then minimizes and clusters any pages that witness under-invalidation bugs. In our evaluation, LQC is able to reproduce 7 human-submitted bugs mined from the Mozilla bug tracker with self-differential testing. LQC is also able to test hundreds of fuzzer candidates per minute in both Chrome and Firefox, finding dozens of bug candidates in each browsers (79 in Chrome and 125 in Firefox). Furthermore, LQC’s bug minimization and clustering passes reduce the number of style elements by 62.60% and 99.32% relative to minimized reports, all while requiring only 14.16% percent of the runtime. Finally, we show that LQC finds 16 novel bugs, across both browsers, that were reported to and accepted by the developers, with 7 already fixed.

Index Terms—Browser fuzzing, layout invalidation, under-invalidation, self-differential testing

I. INTRODUCTION

Web browsers are central to modern computing, hosting applications that correspond to over 80% of daily work in some organizations [1]. Browser makers therefore invest heavily in browser correctness, [2]–[7], including through fuzzing [8], which uses a large number of randomly generated inputs to identify bugs in the browser.

Browser developers use fuzzers to find both security and correctness bugs. On the security side, fuzzers such as Domato, Grammarinator, Fuzzowski, and MFFA [9]–[12] detect crashes in parsers, network libraries, and media engines, which are both exposed to untrusted input and must maximize performance. On the correctness side, fuzzers such as R2Z2 [13], Janus [14] and Metamong [15] use screenshots to detect browser rendering inconsistencies between different versions of the same browser, or between renderings of the different pages in the same browser, that may indicate a regression or rendering bug. However, all of these approaches struggle to precisely identify *under-invalidation bugs*, a subtle yet critical type of correctness bug in browser layout engines.

Invalidation refers to the process by which the browser updates internal data structures in response to style changes [16]. Since style changes are frequent (often every frame, 60 or even 120 times per second, in response to animations), substantial research has been devoted to designing efficient invalidation procedures [17], [18]. Generally, browsers attempt

to update *as little* of their internal data structures as possible; however, updating *too little* of this internal state leads to an incorrect rendering that mixes fresh (updated) and stale (not updated) data. Updating exactly the right set of fields requires reasoning about transitive dependencies between layout algorithms, reasoning that is difficult enough that public bug trackers for Chrome, Firefox, and Safari’s layout engines contain numerous under-invalidation bugs [19]–[24]. Modern invalidation algorithms like LayoutNG [25] are purposefully designed to reduce the risk of under-invalidation, while some promising algorithms [26] have failed precisely because under-invalidation bugs proved difficult to find and fix.

To combat these issues, this paper introduces a new oracle that detects under-invalidation bugs via *self-differential testing*, where either a large or small change results in the *same* web page, and where differences thus indicate under-invalidation, and validate this oracle on bugs mined from real-world browser bug databases. We extend this basic idea into LQC, a fuzzer that uses self-differential testing to detect under-invalidation bugs using random generation and minimization of complex web pages and web page changes, and leveraging automated bug clustering to reduce the bug review burden. We demonstrate that LQC is effective at both rapidly finding under-invalidation bugs (79 in Chrome and 125 in Firefox) and minimizing and clustering them, reducing review load by 99.74%. 16 bugs found by LQC were reported to Chrome and Firefox, all accepted by developers and 7 already fixed. LQC is now used as part of Mozilla’s fuzzing workflow to find under-invalidation bugs during development.

This paper makes the following contributions:

- A new class of bugs, under-invalidation bugs, that can be efficiently detected using *self-differential* testing;
- LQC, a targeted fuzzer for under-invalidation bugs that generates web pages and web page changes, detects under-invalidation bugs with self-different testing, and minimizes and clusters the resulting bugs to reduce review load.
- An evaluation of LQC and its major components on both Chrome and Firefox, measuring bug discovery, minimization, and clustering with both direct and ablation studies.

We release LQC together with evaluation artifacts, test cases, and benchmarks at the following URL: [**TODO: URL**].

II. BACKGROUND, RELATED WORK, AND MOTIVATION

This section introduces web browsers and layout, invalidation and under-invalidation bugs, and the challenges of fuzzing browser layout engines.

A. Web Browsers

Web browsers are among the most widely used and economically important software systems. A browser must load applications and content over the network; execute applications and react to user input; and render the application user interface to the screen. The rendering step is performed by the browser’s *rendering engine* [16]; Fig. 1 is a high-level diagram of this rendering pipeline. The main production browser engines are Chrome’s Blink engine [27], Firefox’s Gecko engine [28], and Safari’s WebKit engine [29], which power both the named browsers and many others, such as Microsoft Edge and Samsung Browser. Other browser-engine projects include Pale Moon, Ladybird, and Servo [30]–[32]. These engines require substantial and sustained investment; for example, Open Web Advocacy estimates that Google provides roughly 90% of Blink development funding and invests around \$1 billion annually in the web platform [33].

This paper focuses on *layout*, which involves computing the size and position of every web page element. Layout is one of the most complex parts of the rendering pipeline [16], and, because browser rendering is performed after every user interaction, it must be highly optimized [17], [34], [35]. Rendering engines achieve high performance via *invalidation*: most user interactions, animations, or APIs only affect a part of the web application’s user interface, and the browser attempts to re-render only that portion of the page, re-using prior rendering results for the rest [16]. For example, suppose a user hovers over a menu, causing a dropdown to appear. The browser must recompute the position of the dropdown and all of the elements within it, but can reuse the size and position of the rest of the page, since they are unaffected by the user action.

Invalidation leads to large speed-ups [17], but risks two kinds of bugs [34], [36]. *Under-invalidation* occurs when the browser fails to recompute changed portions of the user interface, causing a partially stale interface to be shown the user, which can confuse the user or even break application functionality. For example, the browser might recompute the dropdown’s visibility status without recomputing its size and position, causing it to appear in the wrong location. Under-invalidation bugs often require a specific constellation of HTML, CSS, and JavaScript code to appear. For example, Chrome bug #40724799, found with LQC and since fixed by the Chrome developers, occurred required three nested elements, with the outermost having a `vertical-lr` writing style the middle one having a `inline-grid` display style, the innermost having a `horizontal-tb` writing style, and the application then setting the white-space style of the middle element to `break-spaces` and `back`. Chrome would fail to recompute an internal property called `padding`, requiring four rendering cycles to correctly compute the size and position of the middle element. Needless to say, constructing such an unlikely sequence by hand is difficult, making fuzzers valuable. Over-invalidation bugs, where the browser recomputes too much of the user interface, are also possible. That said, while these can slow rendering, they do not cause correctness

issues and aren’t a focus of this paper [25].

B. Browser Fuzzing

Fuzzers have a long history in browser testing, especially due to browsers’ exposure to untrusted data over the network. JavaScript-engine fuzzers such as CodeAlchemist and DIE generate complex programs to expose crashes and security vulnerabilities [37], [38]. Grammar-based fuzzers such as Domato [9], by contrast, generate structured inputs to exercise deeper portions of the browser pipeline, while other browser fuzzers target specialized attack surfaces such as WebGL or WebAssembly [39], [40]. These approaches focus on detecting crashes, which often indicate a security vulnerability. For example, RTFuzz mutates render-tree state, similar to LQC, but primarily searches for crashes [41].

A newer collection of fuzzers instead target rendering correctness. For example, R2Z2, Metamong, Janus [13], [14] generate structured web pages and compare their differential rendering either across browsers or across browser versions. This differential testing approach can detect cross-browser compatibility issues, but are challenging to scale because different browsers or versions can differ for legitimate reasons, including different platform libraries, different user interface elements, or different supported features, among others. Under-invalidation bugs are especially ill-suited to a differential testing approach, because browser implement different invalidation algorithms. On the other hand, directly testing a browser layout against a known-good or reference implementation is not possible, because no browser is known to generate correct output in all cases and no reference implementation exists.

Under-invalidation bugs should be found by fuzzing.

Existing browser fuzzers, which focus on differential testing, cannot easily find under-invalidation bugs. LQC uses self-differential testing instead.

III. CHALLENGES AND SOLUTIONS

To address the limitations of conventional browser fuzzing for layout under-invalidation bugs, we introduce LQC: a fuzzer that finds, minimizes, and groups under-invalidation bugs in product web browser layout engines. LQC’s design is shaped by three key challenges.

A. Challenge 1: Defining a Correctness Oracle

The central challenge for layout correctness is in constructing an oracle that detects layout bugs.

a) *No obvious failures*: The web platform guarantees that web page layout is always a successful, atomic operation. There are neither failures nor intermediate states to observe. Crash-based and assertion-based fuzzers are thus insufficient to detect under-invalidation.



Fig. 1. A high-level diagram of the browser rendering pipeline. Of these steps, layout is most complex, as it instantiates the complex semantics of the CSS standard. The pipeline is rerun on every user interaction or animation, and invalidation is the main way browsers achieve high performance. Image reused with permission.

b) No ground truth: There is also no external ground truth to determine correct layout. Different browsers use different default fonts, different text layout libraries, and different user interface components. They also support different sets of web platform features, and use different implementation internals; for example, Firefox performs layout in sixtieths of a pixel while Chrome uses sixty-fourths of a pixel [?]. Plus, some aspects of layout, like table layout, are not fully specified. When two browsers disagree, this does not guarantee that either one is wrong. The Janus fuzzer also faced this issue [?], and compensated by using a weaker oracle: two different browsers should either both change or both not change their rendering in response to a user action. However, mistakes in invalidation often result in *partial* updates, making this weaker oracle insufficient to catch under-invalidation bugs.

c) Rapid change: Browser rendering engines also rapidly implement new features, making regression testing challenging. The R2Z2 fuzzer also faced this issue [?], and used the Web Platform Tests cross-browser test suite to differentiate between new features and new bugs. However, this is insufficient to detect new features that are implemented incorrectly, which is where detecting under-invalidation bugs is most valuable, an issue noted by the R2Z2 authors.

d) Self-differential testing: LQC addresses this challenge using self-differential testing oracle. The same HTML and CSS may be laid out correctly when the browser computes layout from scratch, but incorrectly when the browser reaches that state through an incremental update. But the web platform standards do not allow for path dependence; the layout of a web page should depend only on the page itself and browser parameters like viewport size. LQC thus compares the browser’s layout of a page after incremental layout to the layout after from-scratch layout. Any difference indicates an under-invalidation bug. This approach compares browser rendering engine against itself, avoiding all sources of cross-browser and cross-version differences.

B. Challenge 2: Triggering Under-Invalidation Behavior

A fuzzer must not only detect when a bug has been triggered but also generate inputs that trigger them. Unfortunately, doing so for under-invalidation bugs is difficult, because these bugs often rely on complex situations to trigger.

a) Representing Changes: Invalidation algorithms consider both the web application state—the DOM, consisting of HTML elements and their CSS properties—and also the changes made to that state since the last layout. Fuzzing for under-invalidation bugs thus requires generating both. LQC

thus generates a tree of HTML elements and, for each element, both a set of initial CSS properties and a set of changes to those CSS properties. Rendering the initial HTML and CSS and then performing the indicated changes can then trigger an under-invalidation bug.

b) Element relations: Under-invalidation bugs also depend on the relative position of elements in an application; a bug might involve an element have a certain CSS property and whose child has a certain other CSS property assigned. Children, grand-children, parents, siblings, and even more distant relationships affect layout and can thus be relevant to invalidation decisions. LQC thus generates large element trees where a multitude of elements with various relations are available.

c) Grammar Coverage: Under-invalidation bugs are most valuable to find during development, when algorithms are easiest to chance. Yet development typically focuses on new and then-little-used features. In the case of layout under-invalidation, this means support for the span of CSS properties is essential. For this reason, LQC is grammar-directed using Google Chrome’s actual CSS properties grammar. The grammar-directed generation is used for both the initial CSS properties and for generating new CSS properties that are then assigned to change the application state.

d) Low Success Rate: Under-invalidation bugs are rare due to the high effort developers already go through to prevent them. This means most bug candidate do not trigger a bug, and makes fuzzing slow. Moreover, some under-invalidation bugs are known issues that are difficult to fix due to the structure of surrounding code; these bugs are less valuable to find than yet-unknown “long-tail” bugs, which are even rarer. To address this low success rate, LQC generates large web application states, with dozens of elements each with hundreds of initial CSS properties and hundreds of property changes.

e) Bug Complexity: While large application states are effective at raising the success rate for bug generation, the resulting application states are too complex for developers to effectively address the bugs found. LQC thus uses minimization to shrink successful bug candidates, removing elements, properties, and property changes, while preserving the candidate’s bug-triggering behavior.

C. Challenge 3: Reducing Report Complexity

Some under-invalidation bugs can be traced to a simple logic error that developers can easily fix. Others can be useful for shaping the direction of work in progress, by identifying unworkable designs and driving the design process.

Others, however, are the consequence of developers previously selecting an unworkable design and now being unable to easily change it. Browser rendering engines are decades-old million-line code-bases, and all three types exist in abundance, but a fuzzer cannot easily distinguish between them.

a) Steering: Layout is complex and developers often work on one component at a time. Finding under-invalidation bugs in that specific component is then especially valuable, since they can be immediately fixed. This requires allowing the developer to influence the search space, such as by indicating CSS properties or CSS property values that the fuzzer should preferentially generate.

b) Summarization: Many fuzzer-generated bug candidates trigger the same underlying bug; tediously examining them all by hand, or even with AI assistance, is likely to be time-consuming. Since similar bug candidates—especially after minimization—likely trigger the same bug, clustering bugs by similarity reduces the review burden. Since under-invalidation bugs often depend on structural relationships in the element tree, the clustering technique should account for these relationships, along with the CSS properties or property values that are present within the cluster.

IV. IMPLEMENTATION

LQC is implemented in Python as an end-to-end, feedback-free fuzzing pipeline for browser layout invalidation. The implementation executes the serialized page in a real browser through Selenium WebDriver, and identifies an under-invalidation failure when the incremental layout version and refreshed layout version disagree. The same pipeline then generates candidate reports, minimizes them, and groups structurally related reports. The following sections describe the implementation: the layout oracle, the generation of web pages and bug candidates, and the clustering of reports.

Fig. 2 summarizes the main stages of the LQC implementation pipeline.

A. Layout Oracle

LQC checks layout correctness using self-differential testing: it tests the browser against itself rather than comparing one browser against another browser or against an external reference implementation. This is the core idea of the oracle. For each test case, LQC asks whether the browser produces the same layout for the same final page state when that state is reached in two different ways.

To validate that this oracle captures real under-invalidation failures, we converted 11 historical browser bug reports into equivalent LQC test cases. Table I decomposes each case into the three components LQC generates: an HTML tree, initial styles, and a style mutation.

This differs from most existing browser fuzzers because LQC does not treat crashes, security violations, API exceptions, or cross-browser differences as its main correctness signal. Crash-oriented fuzzers can find memory-safety and stability bugs, but an under-invalidation bug may leave the browser running normally while still displaying a stale layout.

Cross-browser rendering fuzzers can find cases where two browsers disagree, but disagreement alone does not show which browser is wrong or whether the problem is caused by incremental layout invalidation. In contrast, LQC compares two layout computations inside the same browser for the same final page state, making the oracle specifically targeted at stale incremental layout version results.

The first execution path is the incremental layout version. LQC loads a test case, applies a style update, and records the resulting layout. This is the point where the layout engine decides which cached layout state must be invalidated after the page changes. The second execution path is a refreshed layout version. After the style update has been applied, LQC forces the same browser to rebuild the page from scratch and compute the layout again from the same final HTML and CSS state creating the correct build of this web page configuration.

These two executions should produce identical results. If the incremental layout version differs from the refreshed layout version, then the browser has produced two different layouts for the same final page state. Because both results come from the same browser, the mismatch is not caused by differences between browser engines, feature support, or rendering policies. Instead, it is evidence that the incremental layout version reused stale layout information and failed to invalidate part of the page correctly.

In practice, the oracle records the position and size of each element after the incremental update and then records the same information again after the refreshed layout version. It compares the position, width, and height for matching elements. If all recorded values match, the test passes. If any element has a different layout between the two paths, LQC reports the test as a layout-bug candidate.

This oracle is important because it gives LQC a correctness signal for bugs that do not cause crashes and do not require another browser to serve as a reference. By comparing the browser’s incremental layout version against its own refreshed layout version, LQC directly targets the under-invalidation errors that conventional fuzzers are not designed to detect.

B. Test Case and Bug Generation

LQC generates test cases that are designed to exercise layout invalidation behavior rather than arbitrary browser behavior. Each test case contains an initial page state and a style update.

Internally, each test case is represented as a `RunSubject`:

- an `ElementTree`, which is a nested representation of the generated DOM;
- a base `StyleMap`, containing declarations applied in the initial document; and
- a modified `StyleMap`, containing declarations installed later by JavaScript.

The `ElementTree` defines the page structure, while the two `StyleMaps` define the base style state and the modified style state. The base style state determines the layout that the browser computes when the page first loads. The modified style state is applied later through JavaScript, forcing the

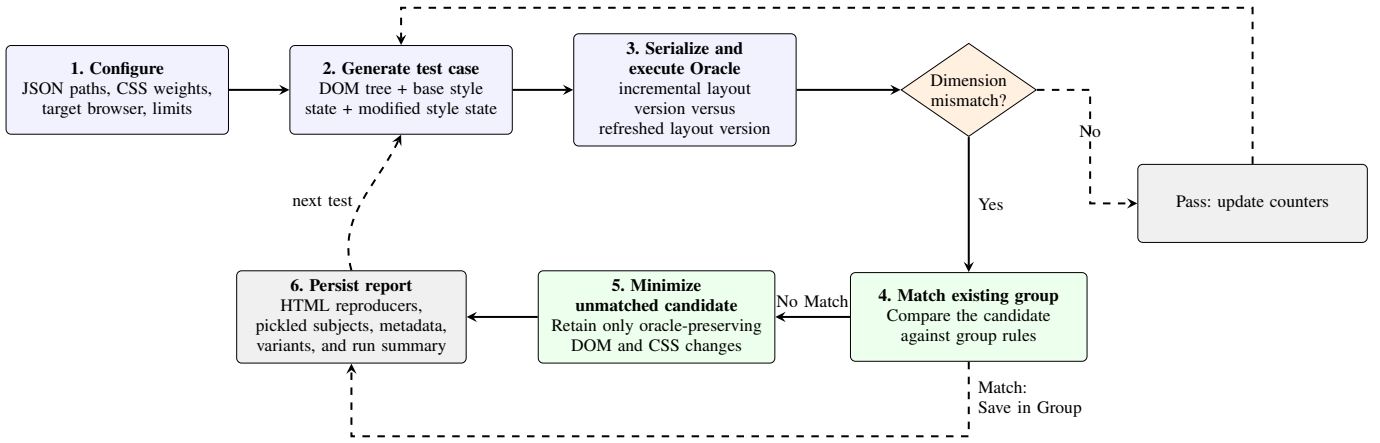


Fig. 2. LQC pipeline. Each test case is tested by a layout oracle. Passing subjects advance the fuzzing loop; candidates are structurally classified, minimized when necessary, and persisted with their reproduction artifacts and cumulative metrics.

browser to use its incremental layout invalidation engine. This test case produces the incremental layout version, which is then later compared to a refreshed layout version of the same final page state.

The generator builds pages from nested elements and text. Nested elements are important because many layout bugs are triggered when there are changes between parents, children, siblings, or descendants. Text is also useful because it introduces content-dependent sizing and positioning. Together, these structures create pages where a single style change can propagate through multiple parts of the layout tree, increasing the chance that a missed invalidation decision becomes observable.

To further increase the likelihood of triggering under-invalidation behavior, LQC biases generation toward CSS properties that are likely to affect layout geometry. Properties such as `display`, grid-related properties, `width`, `height`, percentage sizes, pixel sizes, and transforms are useful because they can change element size, position, layout mode, or parent-child layout relationships. These properties are more likely to require recomputation than purely visual properties, so they give the oracle more meaningful state changes to check.

The generator also supports weighted property selection. These weights allow an experiment to increase the probability of selecting properties that are expected to stress layout invalidation, while reducing or disabling properties that mostly add noise. This changes the distribution of test cases so that more executions target the kinds of mutations that are likely to expose stale layout results.

This design addresses the second challenge by making generation invalidation-aware. Rather than producing only valid HTML and CSS, LQC produces pages with layout-affecting style updates. As a result, each test case is more likely to exercise the browser’s incremental layout logic and more likely to produce a bug candidate when that logic fails to invalidate stale geometry.

C. Minimization and Structural Clustering

Candidate pages are usually much larger than the condition required to reproduce a mismatch. LQC applies delta debugging to reduce a bug-triggering test case while preserving the oracle result. For each proposed simplification, it deep-copies the current subject, executes the candidate in the target browser, and retains the copy only if the result remains a bug. Every accepted reduction becomes the new subject, so later reductions are tested against the smallest subject found so far. The delta-debugging process uses the following proposal families in order:

- 1) remove each element, including its associated base and modified style entries;
- 2) remove every style map for one element at a time;
- 3) remove individual base style state or modified style state declarations;
- 4) move one modified declaration into the base map, thereby testing whether that declaration must be an incremental change rather than merely being present in the final state;
- 5) apply readability-oriented transformations, including minimum box sizes, background colors, short identifiers, and an optional in-page control overlay.

The fourth step in minimization, implemented by `Minify_MoveStyleChangesToFirstLoad`, is particularly important for this bug class. If moving a declaration to the initial state preserves the mismatch, it is not necessary for the incremental transition and can be removed from the set of modified properties. Conversely, declarations that cannot be moved identify the state change that is more likely to matter to invalidation.

As previously explained a test case is saved as a run subject which represents the dom tree and the attached base modified style this structure is used by the clustering algorithm. The first node with modified style state is used as the start node because it anchors the tree comparison relative to this modified node. When merging the merged tree represents

TABLE I
COMPONENTS OF CONVERTED BROWSER BUG TEST CASES.

Case	HTML tree	Initial styles	Style mutation
166194	div > (text + div > div > text) + div	outer, inner, after: clear:both; action, text: float:left; middle: display:none; margin-bottom:10em	middle.display = inline-block
648383	div + div + div > div	first, resizeMe, third: float:left; resizeMe: width:100%	resizeMe.height = 100px
1733276-2	div > (text + div)	container: width:100px; test: display:none; margin-left:150px	test.display = inline
1733276-3	div > (text + div)	container: width:100px; test: display:none; padding-left:150px	test.display = inline
Overflow/move	div > div > div	absolute: position:absolute; left:200px; top:0; overflow:hidden; inner2: height:1000px; overflow:auto	left = 100px; top = 100px
Letter spacing	text + pre	pre: display:inline; letter-spacing:1em	width = 50px; layout; width = auto
Table display	table > tr > td > div	td: width:100px; height:100%; element: display:block; height:100%	element.display = table
Scroll area	div > (div + div)	scrollArea: 200px x 200px; overflow:scroll; abspos: position:absolute	scroll = (50,50); width = 220px
Grid/table	div > (div + (div > div) + div)	five: display:table-column; grid-auto-columns:20px; four: display:table; two: display:table-row	five.display = grid
Multicol/flex	div > (div + div > (div + div))	multicol: columns:4; height:100px; flexbox: display:flex; target: height:50px; contain:size	target.height = 900px
Abspos inline	text + span > div	span: position:relative; abspos: position:absolute	vertical-align = top

Note: We took test cases from browser bug reports and converted them into equivalent LQC test cases. These conversions show that, as long as LQC can generate an HTML tree, its initial styles, and a style mutation, it can generate test cases that reproduce these bugs. Only declarations relevant to each mutation are shown.

the DOM structure relative to this anchor: it compares the modified node and its ancestor/descendant structure, rather than depending on that node’s absolute location in the full page tree. This emphasizes a smaller local update pattern to compare with across different test cases.

A group stores a merged representative tree and an extracted JSON rule. To form a candidate rule, the implementation merges two trees. Equal nodes and values are retained. Discrepancies in values are represented by the wildcard value `diff`, meanwhile discrepancies in structure are deleting. The resulting merged tree then has a rule extracted from it which consists of an ordered HTML pattern with the attached base style state and modified style state. Matching checks both components: the incoming subject must contain the ordered structural pattern and at least one matching element must have the required base style state and modified style state declarations.

Before accepting either a match or a new group, the rule is validated against the safe repository. The safe repository is a small repository of known-safe subjects. It generates and executes subjects until the configured safe directory contains a set of bug free `RunSubject` objects. A rule is rejected if it matches any safe samples. This is a guard against a broad structural description that would classify ordinary pages as bug

instances.

If an incoming bug matches an existing rule, the runner skips the full reduction and saves it as another instance under that group. Otherwise, it minimizes the bug first. The final minimized subject is then compared against standalone reports. Matching a standalone report promotes the two reports into a newly created `bug-group-*` directory and writes the extracted rule. Group artifacts are recomputed so the merged representative and rule correspond to the instances currently stored in that directory. The sorting time is measured separately and added to the run summary alongside minimization time; this makes the cost and benefit of grouping measurable in Section V. Each saved report is placed in either a standalone `bug-*` directory or a child directory of a `bug-group-*` directory.

V. EVALUATION

Our evaluation of LQC is guided by these fundamental questions:

RQ1: Is LQC at triggering an under-invalidation bugs?

RQ2: Do minimization and clustering reduce report complexity?

RQ3: Does LQC find confirmed under-invalidation bugs?

TABLE II
BROWSER EVALUATION BENCHMARKS.

Browser	Rendering Engine	Driver
Chrome	Blink	ChromeDriver
Firefox	Gecko	GeckoDriver

TABLE III
BUG-DISCOVERY RESULTS ACROSS BROWSER AND CONFIGURATIONS.

Browser	No weights			Weights		
	Speed	Bugs	Rate	Speed	Bugs	Rate
Chromium	196.8	105	0.889%	558.0	100	0.299%
Firefox	495.9	63	0.212%	492.8	137	0.463%

Note: Runs use a 60-minute execution window. Speed is measured in executed tests per minute; rate is bugs divided by executed tests.

Benchmarks: Table II summarizes the browser targets evaluated by LQC. Our evaluation uses Chromium, which exercises the Blink layout engine, and Firefox, which exercises the Gecko layout engine. LQC uses ChromeDriver and GeckoDriver, respectively.

Experiment Setup: Because no existing fuzzer specifically targets under-invalidation bugs, our evaluation focuses on LQC’s efficiency and quality of the results it produces. For RQ1, we run each browser with and without property weights for 60 minutes. We save several characteristics of these runs including speed(number of tests ran per minute), bugs found, rate of bug discovery, and the overall execution time. During the 60 minute window every minute a snapshot of these metrics were saved to accurately track the metrics across time. This data is represented in Fig. 3, which shows the bugs detected over execution time, and Table III, which showcases metrics at the end of the 60 minute window. For RQ2, the evaluations compares Firefox runs with sorting enabled and disabled over a nine hour execution. We measure the number of css properties in the original reports, minimized reports and clustered reports, cumulative post-processing time of minimization and clustering, and execution time. Measuring total css properties takes into account only the bugs that are within a bug group as to not inflate the total css properties of grouped or minimized bugs that were never grouped. For RQ3, we report the status of confirmed bugs and use report groups and single bugs to describe the diversity of the report repository.

A. RQ1: Finding Under-Invalidation Reports

RQ1 establishes whether LQC can repeatedly drive independent browser engines into under-invalidation behavior. Property weighting is a secondary comparison: it biases generation toward selected CSS properties, while the unweighted configuration samples from the same property space without that bias. Fig. 3 shows that reports continue to accumulate during the 60-minute window, and Table III distinguishes report count from throughput and report-triggering rate.

Outcomes: Every configuration finds dozens of reports within 60 minutes: Chromium finds 105 reports without prop-

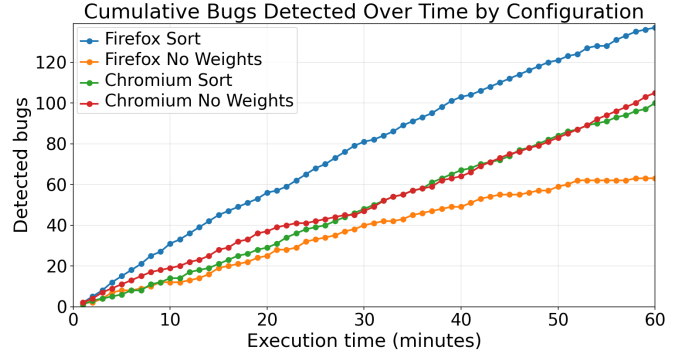


Fig. 3. Cumulative bug reports for the weighted and no-weight configurations over the 60-minute evaluation window.

erty weights and 100 reports with property weights, while Firefox finds 63 and 137 reports, respectively. The nonzero rates in both Blink and Gecko show that LQC reaches the target behavior in two independent layout engines, rather than depending on one browser-specific implementation.

The weighted comparison explains how configuration choice affects that ability. In Firefox, weighting raises the report count from 63 to 137 and the report-triggering rate from 0.212% to 0.463%, while throughput remains nearly unchanged (495.9 versus 492.8 tests per minute). In Chromium, weighting increases throughput but does not improve discovery rate: the unweighted configuration produces more reports (105 versus 100) and a higher report-triggering rate (0.889% versus 0.299%). Thus, higher throughput alone does not explain the usefulness of property weights depends on the browser engine and the properties they emphasize.

RQ1: LQC finds under-invalidation reports in both Chromium and Firefox within one hour. Property weights improves discovery rate, but their benefit is browser-dependent rather than universal.

B. RQ2: Reducing Developer Triage Work

Finding a bug is most useful when it is easy to understand and analyze. RQ2 asks whether minimization and clustering reduce that burden, in addition it is important to examine the efficiency impacts. Minimization removes unnecessary CSS style elements from a report. Clustering recognizes repeated reports and retains a shared rule and minimized representative, which can in turn be used to avoid full minimization for every bug found. We therefore evaluate two linked outcomes: how much smaller the developer-facing reports become, and whether the extra processing reduces fuzzing throughput. Fig. 4 measures the former; Fig. 5 measures the latter.

Outcomes: The report-size results show that the clustering and minimization pipeline meaningfully changes the volume of a bug analysis needed. Minimization reduces the style elements in grouped reports by 62.60% relative to the original reports. Combining minimization with clustering reduces the style-element volume by 99.74% relative to originals and

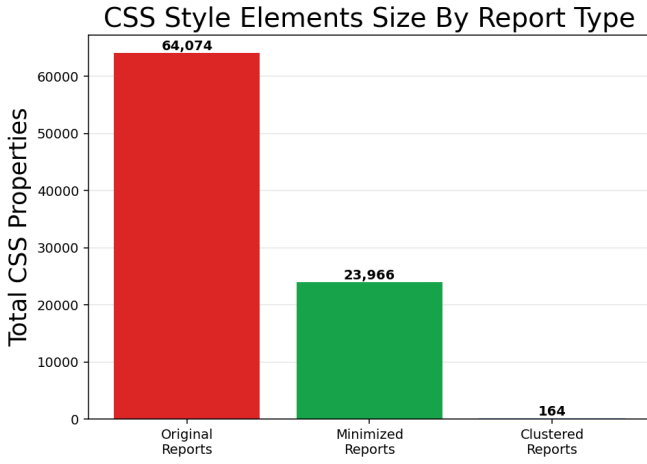


Fig. 4. Total CSS style elements in grouped Firefox reports before minimization, after minimization, and after clustering.

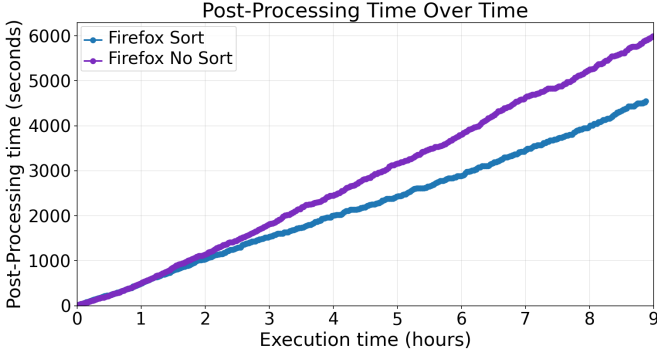


Fig. 5. Cumulative minimization and clustering time for Firefox sorting and no-sorting configurations.

by 99.32% relative to minimized reports. Clustering removes a significant amount of repeated work across reports that represent the same behavior. LQC retains the group’s shared rule and minimized representative instead of independently minimizing and inspecting the new report. Developers can therefore focus on one representative report per behavior rather than repeatedly examining equivalent reports.

In addition the minimization-and-sorting pipeline accounts for only 14.16% of the Firefox sorting run’s runtime. Moreover, specifically the sorting of bug reports into bug groups does not introduce an overhead; instead, it reduces post-processing cost when compared to simply minimizing each bug. At the latest common snapshot, the sorting configuration executes $\sim 130,640$ test cases, compared with $\sim 126,210$ without sorting, while spending $\sim 4,550$ seconds on minimization and sorting rather than $\sim 5,890$ seconds without sorting. When a report matches an existing group, LQC can reuse that group’s representative instead of fully minimizing the new report. Consequently, sorting reduces cumulative post-processing time by $\sim 22.6\%$ and enables $\sim 4,430$ additional tests to run.

TABLE IV
STATUS OF BUG REPORTS SUBMITTED TO CHROME AND FIREFOX.

Browser	Bug ID	Brief description	Status
Chrome	1137427	Repeated white-space application failure	Fixed
Chrome	1166887	Dynamic table-caption vertical order swap	Fixed
Chrome	1189755	Fixed-position grid layout under-invalidation	Fixed
Chrome	1189762	None-to-inline-grid display invalidation	Open
Chrome	1190220	Grid layout page crash	Fixed
Chrome	1219641	Inline-grid list-item text-indent invalidation	Open
Chrome	1224721	Grid layout under-invalidation report	Fixed
Chrome	1229681	Grid child display-none invalidation	Open
Firefox	1721719	Table row-group height inconsistency	Open
Firefox	1724982	Table min-height overflow discrepancy	Open
Firefox	1724991	Dynamic break-spaces invalidation inconsistency	Open
Firefox	1733276	Display-inline margin line-wrapping inconsistency	Open
Firefox	1735376	Grid table-row height inconsistency	Closed
Firefox	1735931	Percent-sized grid-item height inconsistency	Closed
Firefox	1748891	Sticky padding scrollbar position inconsistency	Open
Firefox	1724999	Layout Quick Check metabug	Open

RQ2: Minimization and clustering make reports substantially easier to dissect while preserving, and in this experiment improving, fuzzing throughput. They reduce both the size of individual reports and the repeated work spent on reports already represented by a bug group.

C. RQ3: Finding Confirmed Bugs

The candidate reports measured in RQ1 are not, by themselves, evidence of distinct root-cause bugs. RQ3 therefore asks whether LQC produces reports that survive external developer review. LQC produced 16 confirmed under-invalidation bugs; Table IV records their current statuses. 7 of these reports were fixed or closed by developers. We also reproduce 7 human-submitted Mozilla bugs, showing that the oracle can reach known under-invalidation behavior.

Clustering provides complementary evidence about report diversity before manual validation. Fig. 7 shows how reports are divided into bug groups and single bugs over time, while Fig. 6 shows the final size distribution of the Firefox groups.

Outcomes: Table IV establishes that LQC’s reports are actionable beyond the experimental oracle. It produced 16 confirmed bugs, and 7 were fixed or closed. These outcomes provide external evidence that the tool reaches real under-invalidation defects rather than only reporting test artifacts.

The clustering results provide evidence that LQC discovers a diverse set of potential bugs rather than merely generating many syntactic variations of the same behavior. The clustering process groups reports with shared behavior while retaining a minimized representative for each group. At the latest common snapshot, the Firefox sorting run produces 565 reports across 46 groups and 37 unmatched reports. The number of groups, each represented by distinct HTML and CSS combinations, indicates that the report repository contains substantial behavioral variation rather than a single bug expressed repeatedly. These counts are evidence of potential diversity, not confirmed root-cause-bug counts; confirmation of novelty comes from the submitted reports reviewed by browser developers in Table IV.

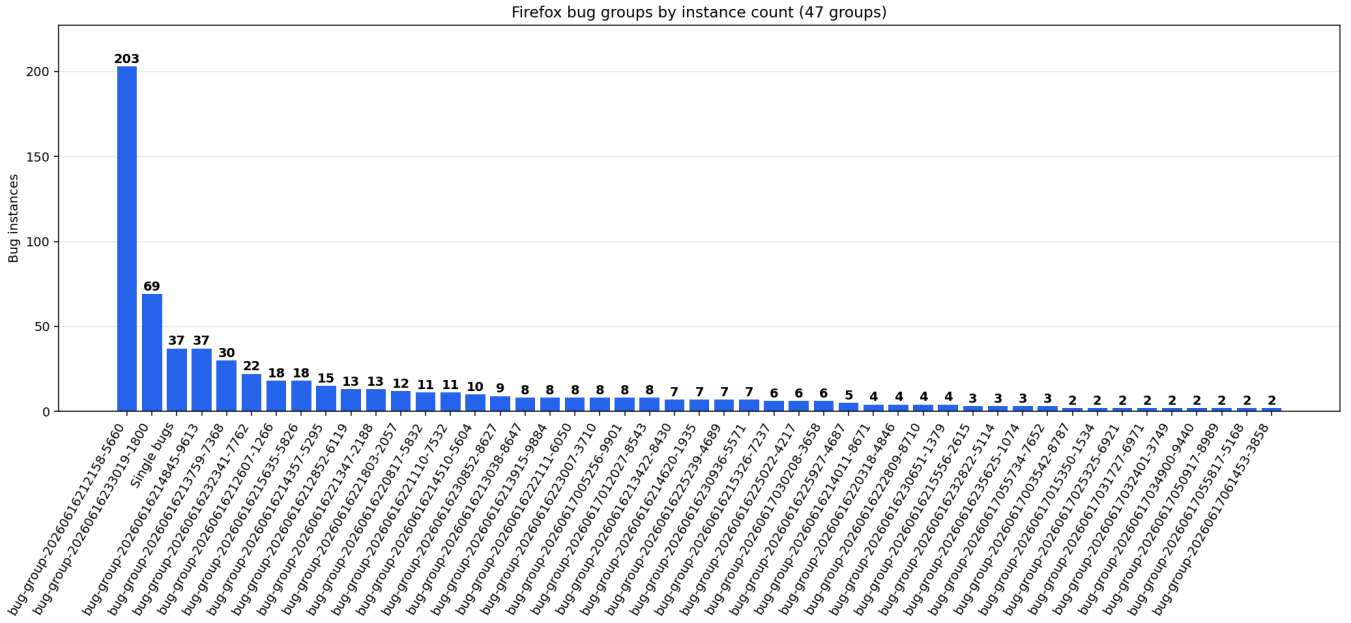


Fig. 6. Firefox bug-group sizes, with all single bugs combined into one bar.

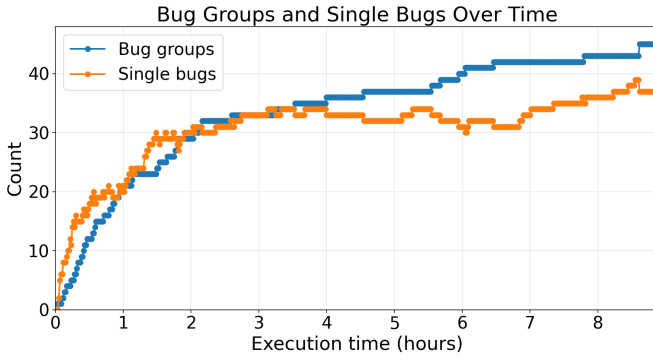


Fig. 7. Bug-group and single-bug counts by total detected reports in the Firefox sorting run.

RQ3: LQC finds confirmed under-invalidation bugs: it produced 16 confirmed bugs, of which 7 were fixed or closed. Clustering helps further separate recurring reports from candidates to make independent validation easier.

VI. DISCUSSION & THREATS TO VALIDITY

A. Runtime Throughput

A limitation of the current workflow is that test generation, minimization, and sorting can reduce overall throughput. Each generated subject may require many browser executions, especially during minimization, where the current approach removes elements and declarations one at a time. This process is effective, but it can be expensive when a subject contains many HTML nodes or CSS declarations.

A more efficient approach could use divide-and-conquer minimization. Instead of testing each element individually, the

minimizer could first try removing larger groups of HTML nodes or CSS declarations. If removing a group fails to preserve the bug, the algorithm could then test smaller groups within that region. This strategy could reduce the number of browser executions needed to produce a compact test case.

Throughput could also be improved by parallelizing generation, minimization, and sorting. For example, candidate bugs could be placed into a queue for minimization while other workers continue generating and testing new subjects. However, this introduces additional challenges, including resource management, synchronization, and avoiding duplicate reports.

This limitation affects the number of tests the tool can execute in a fixed amount of time. However, it does not invalidate the results, since the current workflow still generates, detects, minimizes, and organizes reproducible browser-layout bugs.

B. Browser Coverage

Another threat to validity is that the results depend on which browsers are supported and tested. Supporting a browser requires browser-specific configuration, a compatible automation driver, and reliable access to layout information used by the oracle. These requirements make it practical to test major browsers with good WebDriver support, but they make it harder to test uncommon browsers, older versions, or browsers with incomplete automation support.

This means that the tool may miss bugs that only appear in less common browser engines or older browser versions. These browsers may contain defects that are not found in Chromium, WebKit/Safari, or Firefox, especially if they receive less testing and maintenance. As a result, the findings should not be interpreted as covering all possible browser implementations.

At the same time, focusing on major browser engines still provides strong practical coverage. The supported Chromium,

Worldwide Browser Usage, May 2026

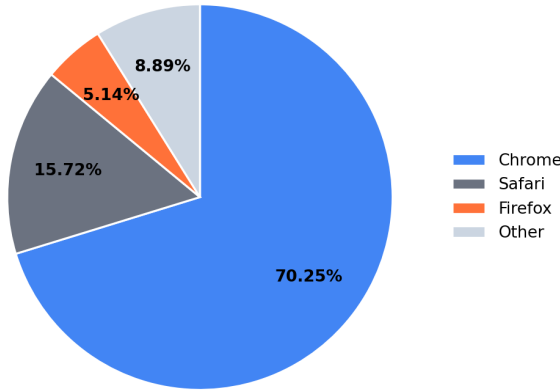


Fig. 8. Worldwide browser usage in May 2026 [42].

WebKit/Safari, and Firefox configurations cover most browser usage. Worldwide in May 2026, Chrome accounted for 70.25% of browser usage, Safari for 15.72%, and Firefox for 5.14%. Together, these browsers accounted for 91.11% of usage [42]. This leaves 8.89% of usage outside the main target of LQC.

Therefore, browser coverage is a limitation of the evaluation, but the current implementation remains useful because it targets the engines with the largest user populations and the most reliable automation tooling.

C. Novel Bug Discovery Rate

The bug detection rate directly affects the efficiency of the tool, since a higher rate produces more candidate bugs per executed test case. However, repeatedly finding the same underlying defect is less useful than discovering distinct bugs. One approach to increasing the bug rate is to identify a stronger static style-weight configuration before a testing run begins. Results from earlier runs can identify HTML structures, CSS properties, and style updates that have produced layout inconsistencies most often. These observations can then be used to assign greater initial weights to promising properties while preserving lower probabilities for less frequently selected properties. Separately, a feedback-driven system could dynamically adjust style-selection probabilities while a testing run is running. It could increase the weights of properties associated with novel layout discrepancies and reduce the weights of configurations that repeatedly produce uninteresting or duplicate results. The system should retain a degree of random exploration so that it does not become overly focused on known bug patterns. Despite this, the current random generation strategy remains useful because it can discover unexpected bug classes without requiring prior knowledge of effective configurations.

VII. CONCLUSION ACKNOWLEDGMENTS

REFERENCES

- [1] F. Montenegro, "The state of workforce security: Key insights for it and security leaders," Palo Alto Networks, Tech. Rep., Jan. 2025.
- [2] Google Project Zero, "Google project zero," Google Project Zero, accessed June 27, 2026. [Online]. Available: <https://projectzero.google/>
- [3] B. Grinstead, C. Holler, and F. Braun, "Behind the scenes: Hardening Firefox with Claude Mythos Preview," Mozilla Hacks, May 2026, published May 7, 2026. [Online]. Available: <https://hacks.mozilla.org/2026/05/behind-the-scenes-hardening-firefox/>
- [4] World Wide Web Consortium, "Web standards," W3C, accessed June 27, 2026. [Online]. Available: <https://www.w3.org/standards/>
- [5] Web Hypertext Application Technology Working Group, "Standards," WHATWG, accessed June 27, 2026. [Online]. Available: <https://spec.whatwg.org/>
- [6] web-platform-tests project, "web-platform-tests documentation," web-platform-tests, accessed June 27, 2026. [Online]. Available: <https://web-platform-tests.org/>
- [7] R. Andrew, "Interop 2026: Continuing to improve the web for developers," web.dev, Feb. 2026, published February 12, 2026. [Online]. Available: <https://web.dev/blog/interop-2026>
- [8] Google, "ClusterFuzz: Scalable fuzzing infrastructure," GitHub, accessed June 27, 2026. [Online]. Available: <https://github.com/google/clusterfuzz>
- [9] I. Fratric, "Domato: A DOM fuzzer," GitHub, 2017, accessed June 27, 2026. [Online]. Available: <https://github.com/googleprojectzero/domato>
- [10] R. Hodován, Á. Kiss, and T. Gyimóthy, "Grammarinator: An ANTLR v4 grammar-based test generator," GitHub, 2018, accessed June 27, 2026. [Online]. Available: <https://github.com/renatahodovan/grammarinator>
- [11] M. Rivas, "Fuzzowski: Network protocol fuzzer," GitHub, 2019, accessed June 27, 2026. [Online]. Available: <https://github.com/nccgroup/fuzzowski>
- [12] MFFA contributors, "MFFA: Media fuzzing framework for Android," GitHub, 2015, accessed June 27, 2026. [Online]. Available: <https://github.com/fuzzing/MFFA>
- [13] S. Song, J. Hur, S. Kim, P. Rogers, and B. Lee, "R2Z2: Detecting rendering regressions in web browsers through differential fuzz testing," in *Proceedings of the 44th International Conference on Software Engineering*, ser. ICSE '22. ACM, 2022.
- [14] C. Zhou, Q. Zhang, B. Qian, and Y. Jiang, "JANUS: Detecting rendering bugs in web browsers via visual delta consistency," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*, ser. ICSE '25. IEEE, 2025.
- [15] S. Song and B. Lee, "Metamong: Detecting render-update bugs in web browsers through fuzzing," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE '23. ACM, 2023.
- [16] P. Panchekha and C. Harrelson, *Reusing Previous Computations*. Oxford University Press, 2024, pp. 443–494. [Online]. Available: <https://browser.engineering/invalidation.html>
- [17] M. Kirisame, T. Wang, and P. Panchekha, "Spineless traversal for layout invalidation," *Proceedings of the ACM on Programming Languages*, vol. 9, no. PLDI, pp. 1791–1813, 2025.
- [18] J. Liu, Y. Chen, E. Atkinson, Y. Feng, and R. Bodik, "Conflict-driven synthesis for layout engines," *Proceedings of the ACM on Programming Languages*, vol. 7, no. PLDI, pp. 132:1–132:22, 2023.
- [19] Chromium, "Issue 497183437," Chromium Issue Tracker. [Online]. Available: <https://issues.chromium.org/issues/497183437>
- [20] —, "Issue 379886422 resources," Chromium Issue Tracker. [Online]. Available: <https://issues.chromium.org/issues/379886422/resources>
- [21] Mozilla, "Bug 1452426," Mozilla Bugzilla. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=1452426
- [22] —, "Bug 539356," Mozilla Bugzilla. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=539356
- [23] WebKit, "Bug 233421," WebKit Bugzilla. [Online]. Available: https://bugs.webkit.org/show_bug.cgi?id=233421
- [24] —, "Bug 97801," WebKit Bugzilla. [Online]. Available: https://www2.webkit.org/show_bug.cgi?id=97801
- [25] I. Kilpatrick and K. Ishii, "RenderingNG deep-dive: LayoutNG," Chrome for Developers, 2021, last updated October 8, 2021. [Online]. Available: <https://developer.chrome.com/docs/chromium/layoutng?hl=en>

- [26] MozillaWiki, “Gecko: Reflow refactoring,” MozillaWiki, 2006, landed December 7, 2006. [Online]. Available: https://wiki.mozilla.org/Gecko:Reflow_Refactoring
- [27] The Chromium Projects, “Blink: Rendering engine,” The Chromium Projects, accessed June 2, 2026. [Online]. Available: <https://www.chromium.org/blink/>
- [28] Mozilla, “Gecko,” Firefox Source Docs, accessed June 2, 2026. [Online]. Available: <https://firefox-source-docs.mozilla.org/overview/gecko.html>
- [29] WebKit, “WebKit,” WebKit, accessed June 2, 2026. [Online]. Available: <https://webkit.org/>
- [30] Pale Moon Project, “Pale moon: Custom, private, open-source web browsing,” Pale Moon, accessed June 27, 2026. [Online]. Available: <https://www.palemoon.org/>
- [31] Ladybird Browser Initiative, “Ladybird,” Ladybird, accessed June 27, 2026. [Online]. Available: <https://ladybird.org/>
- [32] Servo Project Developers, “About servo,” Servo, accessed June 27, 2026. [Online]. Available: <https://servo.org/about/>
- [33] Open Web Advocacy, “Break google’s search monopoly without breaking the web,” Open Web Advocacy, Apr. 2025, published April 23, 2025. [Online]. Available: <https://open-web-advocacy.org/blog/break-googles-search-monopoly-without-breaking-the-web/>
- [34] M. Kosaka, “Inside look at modern web browser, part 3,” Chrome for Developers, 2018. [Online]. Available: <https://developer.chrome.com/blog/inside-browser-part3>
- [35] MDN Web Docs, “Web performance,” MDN Web Docs, accessed June 2, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/Performance>
- [36] Mozilla, “Performance best practices for firefox front-end engineers,” Firefox Source Docs, accessed June 2, 2026. [Online]. Available: <https://firefox-source-docs.mozilla.org/performance/bestpractices.html>
- [37] H. Han, D. Oh, and S. K. Cha, “CodeAlchemist: Semantics-aware code generation to find vulnerabilities in JavaScript engines,” in *Proceedings of the Network and Distributed System Security Symposium*, ser. NDSS ’19, 2019.
- [38] S. Park, W. Xu, I. Yun, D. Jang, and T. Kim, “Fuzzing JavaScript engines with aspect-preserving mutation,” in *Proceedings of the IEEE Symposium on Security and Privacy*, 2020.
- [39] H. Peng, Z. Yao, A. Amiri Sani, D. J. Tian, and M. Payer, “GLeeFuzz: Fuzzing WebGL through error message guided mutation,” in *Proceedings of the USENIX Security Symposium*, 2023.
- [40] O. Draissi, T. Cloosters, D. Klein, M. Rodler, M. Musch, M. Johns, and L. Davi, “Wemby’s web: Hunting for memory corruption in WebAssembly,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, 2025.
- [41] Y. Zeng, Y. Wu, X. Lu, and C. Zhang, “RTFuzz: Fuzzing browsers via efficient render tree mutation,” *Computers & Security*, vol. 161, p. 104756, 2026.
- [42] StatCounter GlobalStats, “Browser market share worldwide,” StatCounter GlobalStats, May 2026, worldwide browser market share, May 2026; accessed June 24, 2026. [Online]. Available: <https://gs.statcounter.com/browser-market-share>